

Detecting Song “Breakout” on Spotify: Predicting Charts Success from Initial Stream Trajectories Utilizing AR/MA and Bayesian Regression

I. Introduction & Literature Review

How can you tell whether a song is going to be a hit after just two or three weeks on the charts? That’s the central question we aim to answer in our project. Most existing work either predicts chart success from static audio features measured at release, or only uses a song’s full streaming history, which is only usable in hindsight. This project proposes a two-stage pipeline that avoids both of these limitations. Given only a song’s first 21 days of daily Spotify stream counts after chart entry, can we predict whether it will ultimately reach the Top 10? Stage one fits an ARMA(1,1) time series model to the early stream trajectory to extract momentum features. Stage two feeds those features alongside audio attributes into a Bayesian logistic regression model that outputs a Top 10 prediction with full uncertainty quantification. To our knowledge, no prior work combines AR/MA trajectory features with Bayesian regression for this early-detection system.

The most closely related existing work is Cabansag and Ntegeka (2025), who train Random Forest and XGBoost classifiers in 2024 U.S. Top 200 data combined with Spotify audio features, achieving macro F1-scores near 0.95 when predicting peak chart tier. Their approach uses a song’s full streaming history and produces hard class labels with no uncertainty quantification. Our project on the other hand addresses both limitations by looking only at the first 21 days and producing posterior probability distributions rather than hard predictions. Shapiro (2015) applies Bayesian Additive Regression Trees (BART) to predict Billboard Hot 100 appearances from static audio and metadata features in the Million Song Dataset, demonstrating that Bayesian methods handle song-attribute interactions well while guarding against overfitting. The model is purely static with no streaming data, and uses pre-streaming era Billboard data. We build on this Bayesian framing by incorporating AR/MA-derived momentum features as dynamic inputs grounded in real observed chart behavior. Peirson and Wassenaar (2019) model daily stream counts on the Spotify Top 200 using point process models, identifying strong weekly periodicity and song-specific momentum patterns, directly validating that Spotify stream trajectories are autocorrelated and structured enough for AR/MA modeling. Where they use time series modeling as an end goal, we use it as a feature extraction step to feed our Bayes prediction stage down the line.

II. ML Methods & Data Collection

Data Collection & Curation

We aggregate two publicly available Kaggle datasets. The primary Dataset, Spotify Charts (Dave, 2022), contains 26.2 million daily chart entries from January 2017 through December 2021 across 73 global regions, with daily stream counts, rank, and trend direction per song. The secondary dataset, Spotify Tracks Audio Features (Pandya, 2023), contains 114,000 tracks across 125 genres with Spotify API-sourced audio features including danceability, energy, valence, tempo, acousticness, instrumentalness, liveness, loudness, and speechiness. We filtered the charts dataset to Top 200 entries only, parsed dates, and built 21-day trajectory windows per song by identifying each song's chart entry date and extracting its first 21 days of daily stream counts. Songs with fewer than 7 days of data were excluded. We joined the two datasets on lowercase song title and primary artist name using an inner join, keeping only songs with both complete stream trajectories and matched audio features. The final cleaned dataset contains 3,253 songs, 879 of which are Top 10 (27%), and 2,374 of which are not (73%).

AR/MA Time Series Modeling

We applied first order differencing to each song's stream trajectory. ACF/PACF analysis on example songs from the dataset confirmed strong lag-1 autocorrelation in the differenced series and near-zero signal at lag 2, justifying our fixed ARMA(1,1) specification. AR(1) was fitted manually via OLS using the standard formula $w = (X^T X)^{-1} X^T y$ with no pre built packages. MA(1) was fitted via statsmodels ARIMA since lagged error terms cannot be observed directly. For degenerate series with constant or near zero differenced values we fell back to AR(1) only. We extracted 6 momentum features per song: ar1_coef, ma1_coef, mean_streams, trend_slope, volatility, and early_late_ratio.

Bayesian Logistic Regression

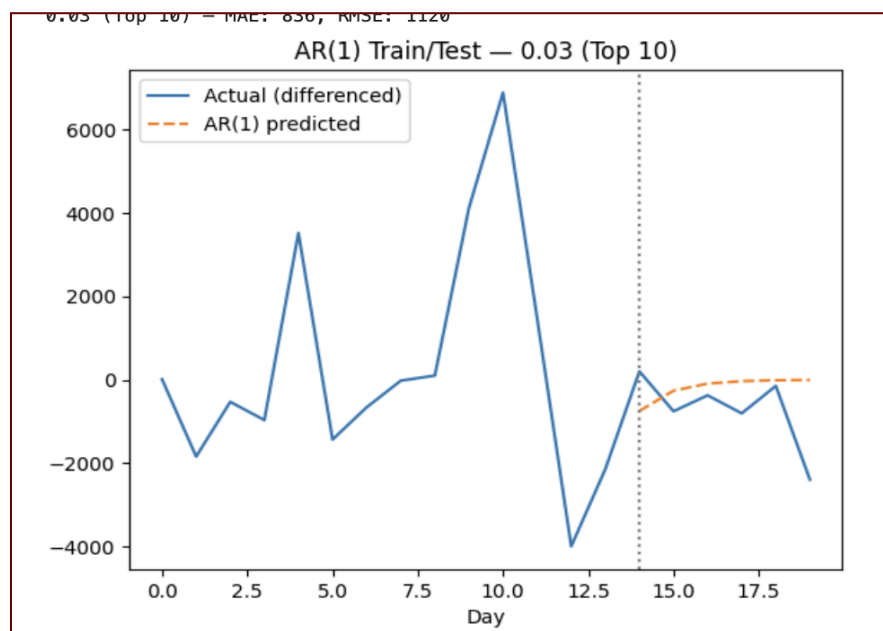
The Bayesian stage uses 12 predictors in total: the 6 trajectory features from Stage 1 plus danceability, energy, valence, acousticness, loudness, and speechiness. We log-transformed mean_streams before scaling because the raw range (1K to 915K) was heavily skewed, and then z-scored all predictors so the priors would be on a comparable scale. We put Normal(0, 2) priors on the slopes, which is a standard weakly informative choice for logistic regression on standardized inputs, and a Student-t(4, 0, 5) prior on the intercept to allow for fatter tails. Sampling used 4 chains of 2,000 iterations each with 500 warmup and thinning of 2, giving 3,000 post-warmup draws. We split the data 80/20 into train and test with a fixed seed (set.seed(42)), yielding 2,602 training songs and 651 for evaluation. The family was Bernoulli logit, which is the natural choice for a binary Top 10 outcome.

III. ML Results & Interpretation

AR/MA Time Series Modeling

Residual diagnostic plots confirmed that ARMA(1,1) captured the autocorrelation structure well across example songs, with residuals behaving like white noise. About 50% passed the ADF

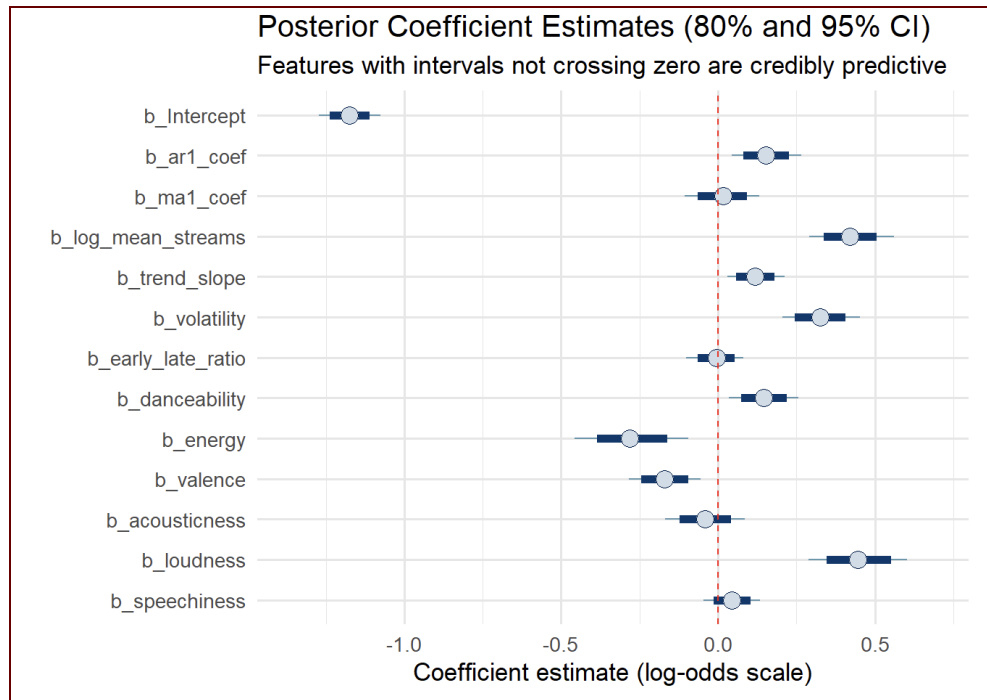
stationarity test after differencing, so there's clearly residual non-stationarity in half the dataset that a fixed-order model can't fully address. The mean AR(1) coefficient across songs was -0.074 , indicating slight mean reversion in the differenced streams, and the mean MA(1) coefficient was -0.611 , which points to strong shock decay: stream spikes tend to dissipate quickly. For example train/test splits, AR(1) showed mean-reverting behavior well but couldn't predict viral spikes, which is a pretty expected limitation of short-window autoregressive models.



(Figure 1): Illustrates the AR(1) model fit on an example Top 10 song, demonstrating that the model captures underlying momentum structure of early stream trajectories despite its inability to predict sharp virality spikes.

Bayesian Logistic Regression

All 13 parameters converged cleanly, with Rhat = 1.00 and both Bulk ESS and Tail ESS above 2,400. Test accuracy came in at 72.7% on the 651 held-out songs. The confusion matrix showed 438 true negatives, 34 true positives, 165 false negatives, and 14 false positives, which works out to 71% precision (when the model flags a hit, it's right about 71% of the time) but only 17% recall (it catches just 17% of true hits). So, the model is conservative which is the expected behavior because of the 73/27 class imbalance.

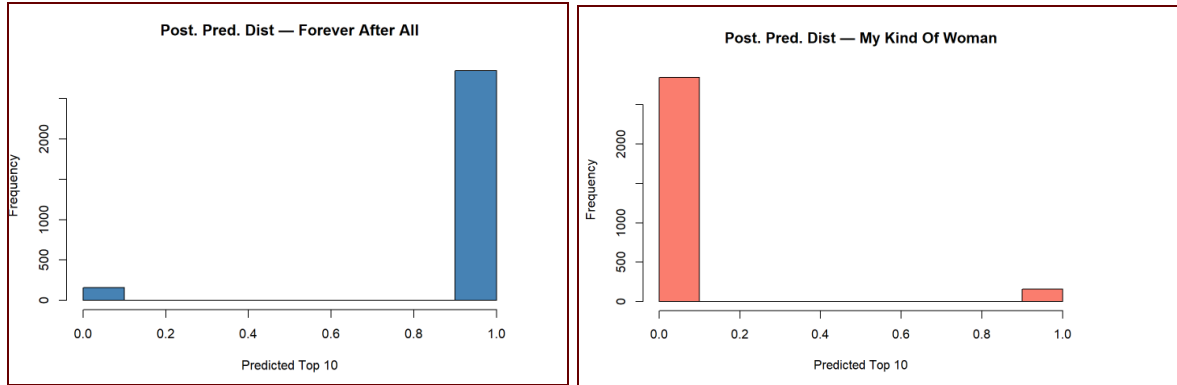


(Figure 2): Shows the full posterior coefficient estimates, providing a visual summary of which features are credibly non-zero and the direction/strength of their effect on $P(\text{Top } 10)$.

Credibly significant predictors (95% CI does not cross zero):

A few predictors were credibly significant, with 95% credible intervals that don't cross zero. `log_mean_streams` was the strongest, at 0.42 with CI [0.29, 0.56], confirming that early stream volume matters more than anything else. Loudness came in at 0.45 [0.29, 0.60], so louder songs are more likely to chart. Volatility at 0.33 [0.21, 0.45] suggests that viral spike behavior in the first 3 weeks is a real signal, not noise. Trend_slope at 0.12 [0.03, 0.21] confirms that an upward early trajectory helps, and `ar1_coef` at 0.15 [0.04, 0.26] shows that the momentum features from Stage 1 add value beyond the audio features alone. On the audio side, energy was credibly negative at -0.28 [-0.46, -0.09] and valence was negative at -0.17 [-0.28, -0.06], meaning lower-energy and more melancholic songs tend to outperform. Danceability was credibly positive but modest at 0.15 [0.03, 0.26]. The features that were not credibly different from zero were `ma1_coef`, `early_late_ratio`, `acousticness`, and `speechiness`.

Looking at individual posterior examples, Forever After All (a true Top 10) had a mean $P(\text{Top } 10)$ of 0.949 with the posterior tightly concentrated near 1.0. My Kind of Woman (a true non-Top 10) had a mean $P(\text{Top } 10)$ of 0.058, also tightly concentrated (near 0.0).



(Figure 3 & Figure 4): Demonstrates the key advantage of the Bayesian approach over a hard classifier, as the model produces a full posterior predictive distribution, showing not just the predicted class but the degree of certainty behind each prediction.

IV. Conclusions & Future Work

The main findings are straightforward. Early stream volume is the most important predictor, more than any audio feature. The trajectory features (volatility, trend slope, ar1_coef) collectively confirm that Stage 1 adds real value beyond what audio attributes alone can tell you. And the Bayesian approach gives calibrated uncertainty rather than just hard class labels, which is more useful for actual decisions about which songs to watch.

A few assumptions are worth calling out. Stationarity is addressed via differencing, but about half of songs still showed non-stationarity by ADF, so a per-song ARIMA(p,d,q) would handle this more cleanly. The Bernoulli likelihood is appropriate for a binary outcome, but the linearity assumption on the log-odds scale is a simplification, and nonlinear interactions between trajectory and audio features almost certainly exist. The 73/27 class imbalance also biases the model toward predicting non-hits, which reduces its practical utility for the thing we actually care about, which is finding breakouts.

There are also some real drawbacks to the setup. The 6% join rate between the two datasets brings in selection bias toward well-known artists since those are the ones with complete audio feature coverage. We're also using global charts only, which ignores regional streaming variation that's probably informative. And the fixed ARMA(1,1) order isn't going to be optimal for every song.

For future work, SMOTE or a class-weighted loss would improve Top 10 recall, which is the main practical weakness right now. Matching through Spotify API track IDs instead of artist/title strings would substantially improve the join rate. A per-song ARIMA(p,d,q) would handle the songs where fixed-order ARMA(1,1) leaves non-stationary residuals, and a Bayesian multilevel

model with artist-level random effects would let us pool information across songs by the same artist instead of treating each track independently.

V. Appendix: Code Implementation

AR/MA – Python (manual OLS implementation)

```
# helper functions
def fit_ar_manual(series, p=1): # manual AR(p) via OLS
    y = np.array(series, dtype=float)
    n = len(y)
    X = np.column_stack([y[p-i-1:n-i-1] for i in range(p)])
    y_target = y[p:]
    return np.linalg.lstsq(X, y_target, rcond=None)[0]

def fit_arma(series): # ARMA(1,1) via statsmodels, fall back to AR(1) if series is degenerate
    try:
        fit = ARIMA(series, order=(1, 0, 1)).fit()
        return fit.arparams[0], fit.maparams[0], fit.resid, fit.aic
    except Exception:
        try:
            fit = ARIMA(series, order=(1, 0, 0)).fit()
            return fit.arparams[0], 0.0, fit.resid, fit.aic
        except Exception:
            return 0.0, 0.0, np.zeros(len(series)), np.nan
```

Bayesian Logistic Regression – R (brms)

```
# fit model
fit <- brm(
  top10 ~ ar1_coef + ma1_coef + log_mean_streams + trend_slope +
    volatility + early_late_ratio +
    danceability + energy + valence +
    acousticness + loudness + speechiness,
  family = bernoulli("logit"),
  data = train,
  prior = manual_prior,
  chains = 4,
  iter = 2000,
  warmup = 500,
  thin = 2,
  cores = 4,
  seed = 42
)
```

References / Bibliography

- [1] Cabansag, I. J., and Ntegeka, P. (2025). "Prediction of Spotify Chart Success Using Audio and Streaming Features." arXiv. <https://arxiv.org/abs/2508.11632>
- [2] Shapiro, H. (2015). "A Bayesian Approach to Understanding Music Popularity." Undergraduate thesis, Duke University.
<https://dukespace.lib.duke.edu/dspace/handle/10161/9747>
- [3] Peirson, A. L., and Wassenaar, E. N. (2019). "Analyzing the Spotify Top 200 Through a Point Process Lens." arXiv. <https://arxiv.org/abs/1910.01445>
- [4] Dave, D. (2022). Spotify Charts [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/dhruvildave/spotify-charts>
- [5] Pandya, M. (2023). Spotify Tracks Dataset [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>